



Week 5 Solution

Question 1

Question

Aging is a solution to the starvation problem.

State True or False.

Answer

True

Solution

Starvation occurs when a process waits for an excessively long time because other processes continuously receive priority.

Aging is a technique used to solve this problem. The priority of a process is gradually increased the longer it waits.

For example:

```
Initial Priority = Low
      ↓
Process keeps waiting
      ↓
Priority increases
      ↓
Eventually gets CPU
```

Thus, aging prevents a low-priority process from waiting indefinitely.

Correct Answer: True

Question 2

Question

Four processes A, B, C, and D have burst times of:

A = 4 ns

B = 5 ns

C = 6 ns

D = 7 ns

They are scheduled using **Round Robin** with a time slice of **2 ns**.

Arrival times:

A = 0 ns

B = 0.1 ns

C = 0.2 ns

D = 0.3 ns

At what time will process **A** complete?

Answer

10 ns

Solution

The time quantum is:

2 ns

The Round Robin schedule proceeds as follows:

Time	Process	Remaining Burst Time
0–2	A	2 ns
2–4	B	3 ns
4–6	C	4 ns
6–8	D	5 ns
8–10	A	0 ns

Process A initially requires 4 ns.

It executes:

First turn: 0–2 → 2 ns completed
 Second turn: 8–10 → remaining 2 ns completed

Therefore, process A completes at:

10 ns

Question 3

Question

Using the same Round Robin scheduling:

A = 4 ns
 B = 5 ns
 C = 6 ns
 D = 7 ns
 Time Quantum = 2 ns

At what time will **all processes complete**?

Answer

22 ns

Solution

The total CPU burst time is:

$$\begin{aligned} &A + B + C + D \\ &= 4 + 5 + 6 + 7 \\ &= 22 \text{ ns} \end{aligned}$$

Assuming no context-switch overhead, the CPU continuously executes processes until all are completed.

Therefore, the total completion time is:

22 ns

Correct Answer: 22 ns

Question 4

Question

Four processes A, B, C, and D have estimated completion times:

$$\begin{aligned} A &= 5 \text{ ns} \\ B &= 7 \text{ ns} \\ C &= 9 \text{ ns} \\ D &= 11 \text{ ns} \end{aligned}$$

They are scheduled using **Round Robin** with a time slice of **2 ns**.

Arrival times:

A = 0 ns
B = 1 ns
C = 2 ns
D = 3 ns

At what time will process **C** complete?

Answer

29 ns

Solution

Using Round Robin scheduling with a 2 ns time quantum, each process receives CPU time cyclically.

The execution proceeds as follows:

0–2	A
2–4	B
4–6	C
6–8	D
8–10	A
10–12	B
12–14	C
14–16	D
16–17	A completes
17–19	B
19–21	C
21–23	D
23–24	B completes
24–26	C
26–28	D
28–29	C completes

Therefore, process C completes at:

29 ns

Question 5

Question

Using the same processes and Round Robin scheduling from Question 4, at what time will all processes complete execution?

Answer

32 ns

Solution

The total execution time required is:

$$\begin{aligned} &A + B + C + D \\ &= 5 + 7 + 9 + 11 \\ &= 32 \text{ ns} \end{aligned}$$

Since the CPU must execute a total of 32 ns of CPU work, and no additional overhead is specified, all processes will finish by:

32 ns

Correct Answer: 32 ns

Question 6

Question

In Question 4, what will be the completion time of process C if **Shortest Job First (SJF)** is used instead of Round Robin?

Answer

21 ns

Solution

The processes are:

```
A = 5 ns, arrival = 0  
B = 7 ns, arrival = 1  
C = 9 ns, arrival = 2  
D = 11 ns, arrival = 3
```

Since SJF is **non-preemptive**, once a process starts executing, it continues until completion.

At time 0, only A has arrived.

Therefore:

```
A executes first: 0-5
```

At time 5, B, C, and D are available.

Their burst times are:

```
B = 7 ns  
C = 9 ns  
D = 11 ns
```

SJF selects the shortest job first:

```
B → C → D
```

Therefore:

A: 0–5
B: 5–12
C: 12–21
D: 21–32

Thus, process C completes at:

21 ns

Question 7

Question

Many more context switches occur when using **non-preemptive Shortest Job First (SJF)** scheduling instead of Round Robin scheduling.

State True or False.

Answer

False

Solution

In **non-preemptive SJF**, once a process gets the CPU, it continues executing until it finishes.

Therefore, context switching generally occurs only when a process completes and another process is selected.

Process A —————> Completes
 ↓
 Process B

In **Round Robin**, however, processes are interrupted after each time quantum.

For example:

```
A → B → C → D → A → B → ...
```

Each quantum can result in a context switch.

Therefore, Round Robin generally causes **more context switches** than non-preemptive SJF.

Hence, the statement is:

False

Question 8

Question

A Round Robin scheduling algorithm with a large time slice will behave similarly to a First-Come-First-Serve (FCFS) algorithm.

State True or False.

Answer

True

Solution

In Round Robin scheduling, a process normally loses the CPU when its time quantum expires.

Suppose the time quantum is very large.

For example:

Process A requires = 5 ns
Time quantum = 100 ns

Process A will finish before its time quantum expires.

The same happens for the next processes.

Therefore, the execution order becomes similar to:

A → B → C → D

This is similar to **First-Come-First-Serve (FCFS)** scheduling.

Thus:

Large Time Quantum
↓
Fewer preemptions
↓
Behaviour similar to FCFS

Correct Answer: True

Question 9

Question

The overhead due to context switches is less if the time-slice duration is small.

State True or False.

Answer

False

Solution

A smaller time quantum means that processes are interrupted more frequently.

For example:

```
Time Quantum = 1 ns
```

```
A → B → C → D → A → B → C → ...
```

Every switch from one process to another requires a **context switch**.

A context switch involves operations such as:

- Saving the current process state.
- Loading the next process state.
- Updating CPU registers.
- Changing execution context.

Therefore:

```
Smaller Time Slice
```

```
↓
```

```
More Context Switches
```

```
↓
```

```
Higher Overhead
```

A larger time slice generally reduces the number of context switches.

Therefore, the statement is:

False

Question 10

Question

Starvation can occur in the **Shortest Remaining Time First (SRTF)** algorithm and **Shortest Job First (SJF)** algorithm.

State True or False.

Answer

True

Solution

Both SJF and SRTF give preference to processes with shorter execution times.

Suppose a long process is waiting:

```
Long Process
  |
  | waiting
  ↓
Short process arrives
  ↓
Short process is selected
```

If short processes keep arriving, the long process may continuously be postponed.

This situation is called **starvation**.

In SJF

A long process may wait because shorter jobs are selected first.

In SRTF

A currently waiting or running long process can be delayed when a new process with a shorter remaining time arrives.

Therefore, starvation can occur in both algorithms.

Correct Answer: True